Лабораторная работа № 6. Геометрические преобразования

Цель работы – освоение навыков обработки растровых изображений с помощью алгоритмических языков программирования путём преобразования координат пикселей.

Краткие теоретические сведения

Положение объектов сцены в 3-мерном евклидовом пространстве может быть задано с помощью 3-х пространственных координат. Функция яркости изображения, воспринимаемого камерой, будет определяться светимостью каждой видимой с позиции камеры точки объекта сцены в направлении камеры. Каждая из этих точек определяется тремя координатами, а функция яркости будет функцией трёх переменных. В связи с особенностями человеческого зрения, состоящими в том, что функция яркости сцены воспринимается практически из одной точки пространства в пределах ограниченного телесного угла, формирующего пирамиду видимости, оптическое изображение 3-мерной сцены можно представить в виде центральной перспективной проекции функции яркости на плоскость, перпендикулярную главной оптической оси. В рамках этой проекционной плоскости любая точка, соответствующая видимой точке сцены, будет задана двумя координатами, а функция яркости физически будет описывать величину потока через данную точку в направлении от видимой точки объекта сцены к камере. Эта величина будет зависеть от светимости точки объекта сцены и расстояния от этой точки до камеры. Преобразование трёхмерных координат объектов сцены в двумерные координаты проекционной плоскости называются проективным преобразованием.

Проективные преобразования

Если координаты трёхмерного объекта задавать в системе координат, начало которой совпадает с точкой, из которой производится наблюдение и которая, соответственно, является центром проекции, главную оптическую ось совместить с OZ, а проекционную плоскость расположить параллельно XOY в *z=const*, то координаты проекции точки объекта сцены (x_1,y_1,z_1) на проекционной плоскости будут (x'_1,y'_1)

(рис. 1). Координаты проекции вычисляются с помощью проективных преобразований: $x_1' = \frac{const \cdot x_1}{z_1}$, $y_1' = \frac{const \cdot y_1}{z_1}$. Если проекционную плоскость расположить в z=1, координаты изображения точки на плоскости будут $x_1' = \frac{x_1}{z_1}$, $y_1' = \frac{y_1}{z_1}$.

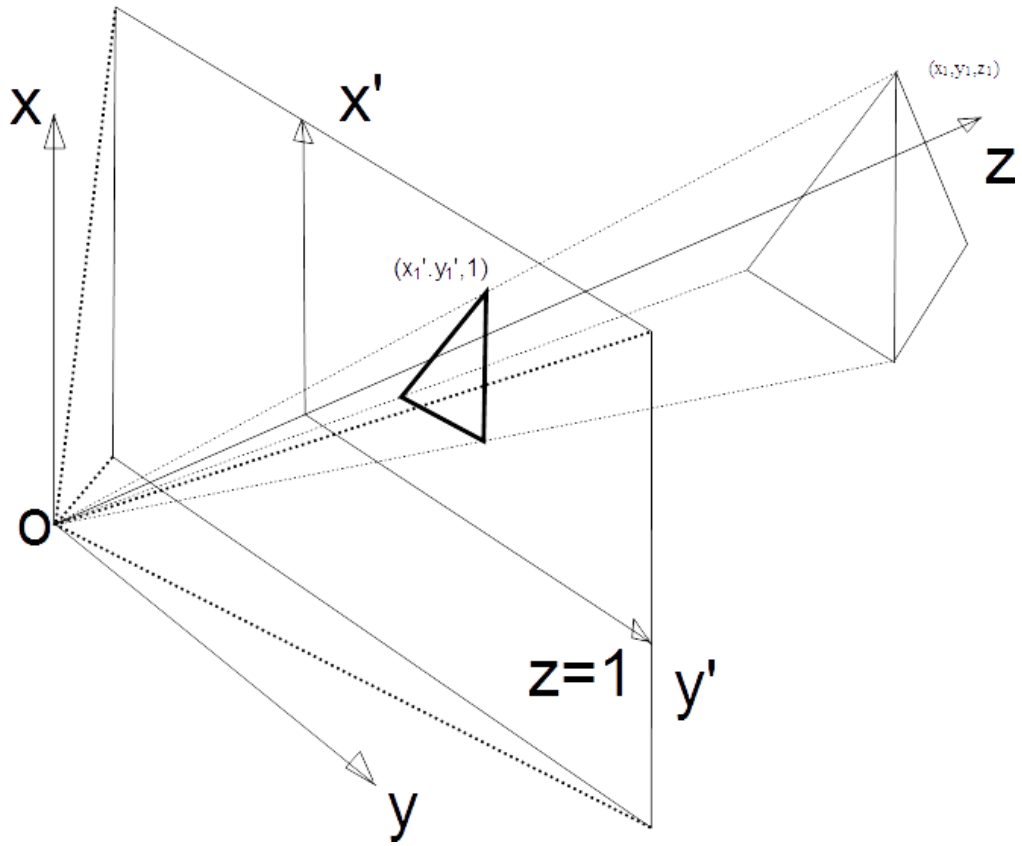


Рис. 1. Построение проекции

Аффинные преобразования

При использовании камеры в конечном счёте функция яркости регистрируется как освещённость чувствительной поверхности фотоприёмника, на которую проецируется оптическое излучение объектов сцены. Освещённость зависит от потока через проекционную плоскость, но координаты функции яркости в плоскости фотоприёмника не обязательно будут соответствовать координатам на проекционной плоскости в связи с тем, что фотоприёмник может быть сдвинут, повёрнут в пространстве и удалён относительно системы координат проекционной плоскости и главной оптической оси (рис. 2). Для приведе-

ния в соответствие выполняются преобразования координат, обеспечивающих перенос, поворот, наклон, масштабирование и отражение изображения. Преобразования такого рода называются аффинными и обладают рядом свойств: любое аффинное преобразование имеет обратное, которое также является аффинным. Произведение прямого и обратного преобразований даёт единичное преобразование, оставляющее все на месте. Аффинное преобразование является самым общим взаимно однозначным отображением плоскости на плоскость, при котором сохраняются прямые линии. Сохраняются также отношения длин отрезков, лежащих на одной прямой (или на параллельных прямых), и отношения площадей фигур. Параллельные прямые переходят в параллельные.

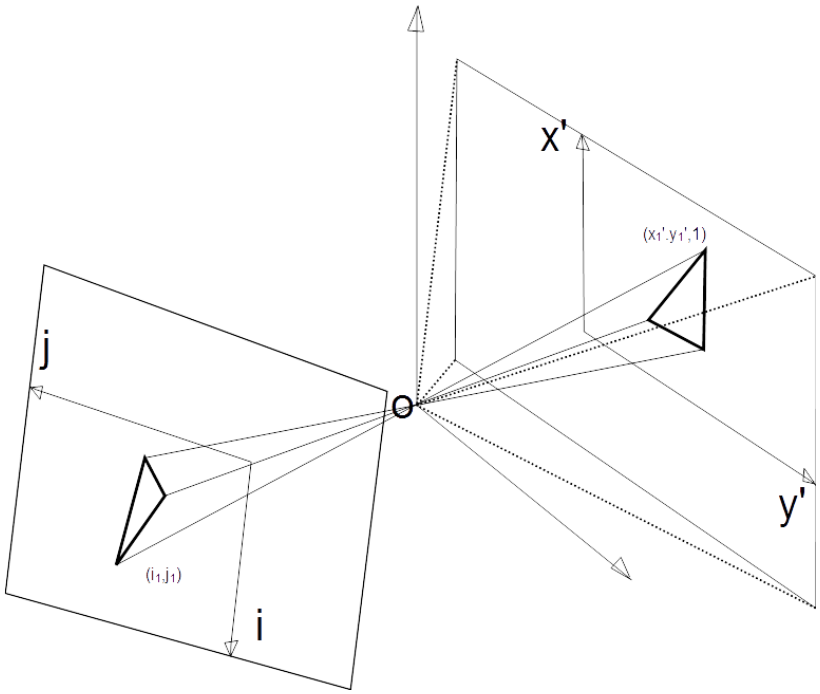


Рис. 2. Возникновение аффинных искажений

Преобразования координат линейны и описываются полиномами первой степени: $i=ax'+by'+c$, $j=dx'+ey'+f$, где i, j – новые координаты; x', y' – старые координаты; a, b, c, d, e, f – параметры преобразования.

Для определения параметров произвольного аффинного преобразования необходимо решить систему из 6 линейных уравнений. Для этого надо задать координаты трёх точек исходного изображения и координаты трёх точек, в которые эти исходные точки должны перейти при преобразовании.

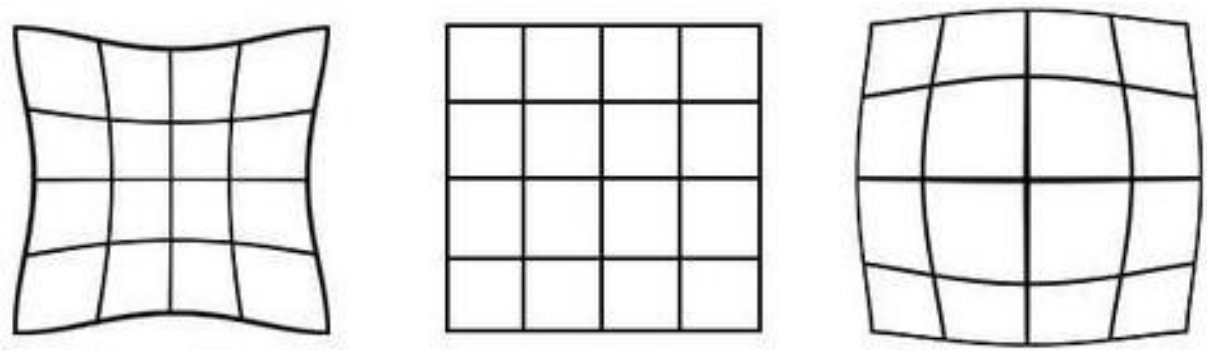
Некоторые частные случаи аффинных преобразований:

– масштабирование по осям: $i=ax'$, $j=ey'$;
– поворот на угол φ: $i=x'\cos\varphi-y'\sin\varphi$, $j=x'\sin\varphi+y'\cos\varphi$;
– перенос: $i=x'+c$, $j=y'+f$;
– скос по x: i=$x'+by'$, $j=y'$, по y: $i=x'$, $j=dx'+y'$;
– отражение: $i=x'$, $j=-y'$ и $i=-x'$, $j=y'$.

Зная параметры аффинного преобразования, можно вычислить непосредственно и параметры обратного преобразования: $x'=a^{-1}i+b^{-1}j+c^{-1}$, $y'=d^{-1}i+e^{-1}j+f^{-1}$, $det=ae-bd$, $a^{-1}=e/det$, $b^{-1}=-b/det$, $c^{-1}=(bf-ec)/det$, $d^{-1}=-d/det$, $e^{-1}=a/det$, $f^{-1}=(cd-af)/det$.

Полиномиальные преобразования

Помимо аффинных геометрических искажений в изображение на поверхности фотоприёмника вносятся искажения, вызванные несовершенством реальных оптических систем и среды распространения оптического излучения. Искажения оптических систем, или аберрации, описываются полиномами более высоких степеней, чем аффинные, и не являются линейными. Например, дисторсия (рис. 3): $i=c_1+x'+k_1(x'^2+y'^2)$, $j=c_2+x'+k_2(x'^2+y'^2)$.



а б в

Рис. 3. Дисторсия: а – подушкообразная; б – отсутствует; в – бочкообразная

В описании дисторсии всего четыре параметра: c_1, k_1, c_2, k_2, для получения их значений нужно взять координаты двух точек на исходном и искажённом изображении и решить систему из четырёх линейных уравнений. Произвольный полином второй степени описывается двенадцатью параметрами: $i=a_{20}x'^2+a_{02}y'^2+a_{11}x'y'+a_{10}x'+a_{01}y'+a_{00}$,

$j=b_{20}x'^2+b_{02}y'^2+b_{11}x'y'+b_{10}x'+b_{01}y'+b_{00}$. Для их определения по искажённой картинке следует определить координаты пар шести точек и решить систему из двенадцати линейных уравнений.

Степень полинома можно и повысить. Количество точек для оценки параметров полинома зависит от количества ненулевых коэффициентов. Для оценки большого количества параметров стараются взять избыточное количество точек и вычислить значения параметров статистическими методами, например методом наименьших квадратов.

Функциональные преобразования

Изображение может быть геометрически преобразовано с помощью любых функций для достижения требуемого эффекта.

Поскольку цифровое изображение дискретно и задано в изолированных узлах, при выполнении преобразования координат на некоторые точки изображения результата может не попасть ни одна точка с исходного изображения. Чтобы избежать пропусков, при обработке цифровых изображений для функции преобразования координат находят обратную функцию, а затем с помощью обратной функции для каждой точки изображения результата определяют значение яркости соответствующей точки исходного изображения.

Например, если $i=x'^2$, то, рассматривая пиксели с координатой $x'=1$ на исходном изображении, будем заполнять пиксели $i=1$ на преобразованном изображении. Переход к следующему пикселю с $x'=2$, приведёт к заполнению пикселей с координатами $i=x'^2=4$ на преобразованном изображении, при этом пиксели с координатами $i=2$ и $i=3$ будут пропущены. Чтобы избежать пропусков, следует найти обратную функцию $x'=i^{1/2}$, перебирать все координаты i на изображении результата и заполнять их пикселями с соответствующими координатами на исходном изображении.

Задание

1. Разработать алгоритм и программу для реализации индивидуального задания. Программа должна выполнять:
   - загрузку изображения из файла;
   - прямые и обратные аффинные преобразования;
   - функциональное преобразование координат;

– сохранение результата в файл.

2. Ответить на контрольные вопросы.
3. Оформить отчёт.

Содержание отчёта

1. Титульный лист.
2. Формулировка задачи.
3. Алгоритм.
4. Программа на алгоритмическом языке программирования.
5. Исходное и преобразованное изображения.
6. Ответы на контрольные вопросы.

Порядок выполнения работы

1. Ознакомление с целью работы, заданием, содержанием отчёта порядком выполнения работы, индивидуальным вариантом, примером выполнения работы, примером оформления отчёта.
2. Ознакомление с теоретическими материалами по теме.
3. Разработка алгоритма в соответствии с вариантом задания. Алгоритм должен предусматривать:

– чтение файла изображения;

– аффинные преобразования изображения указанным способом;

– восстановление исходного изображения из преобразованного путём обратного преобразования;

– нелинейные преобразования указанным способом;

– запись преобразованного изображения в файл.

4. Разработка и отладка программы для решения задачи.
5. Ответы на контрольные вопросы.
6. Оформление отчёта.

Индивидуальные задания

Каждый вариант задания (табл. 1) включает в себя два аффинных преобразования и функциональное преобразование координат, выраженное нелинейной зависимостью.

Таблица 1

Варианты индивидуальных заданий

№	Аффинные преобразования	Функция преобразования
1	Масштабирование, поворот	$i = \frac{x'^3}{1+x'^2}, j = y'$
2	Масштабирование, перенос	$i = 2^{x'}, j = y'$
3	Масштабирование, скос	$i = x'^{\frac{3}{5}}, j = y'$
4	Масштабирование, отражение	$i = exp(x'), j = y'$
5	Поворот, масштабирование	$i = sinh(x'), j = y'$
6	Поворот, перенос	$i = asinh(x'), j = y'$
7	Поворот, скос	$i = \sqrt{exp(x')}, j = y'$
8	Поворот, отражение	$i = x'^{\frac{5}{3}}, j = y'$
9	Перенос, масштабирование	$i = \frac{1}{exp(-x')}, j = y'$
10	Перенос, поворот	$i = x'^{\frac{3}{7}}, j = y'$
11	Перенос, скос	$i = x'^{\frac{7}{3}}, j = y'$
12	Перенос, отражение	$i = \frac{x'^2}{1+x'}, j = y'$
13	Скос, масштабирование	$i = cosh(x') - 1, j = y'$
14	Скос, поворот	$i = acosh(x' + 1), j = y'$
15	Скос, перенос	$i = log_2(x'), j = y'$
16	Скос, отражение	$i = ln(x'), j = y'$
17	Отражение, масштабирование	$i = \sqrt{x'}, j = y'$
18	Отражение, поворот	$i = x' \cdot y', j = y'$
19	Отражение, перенос	$i = x' - exp(-y'), j = y'$
20	Отражение, скос	$i = \frac{x'^2}{y'}, j = y'$

Контрольные вопросы

1. Какие преобразования координат происходят в процессе формирования на поверхности фотоприёмника двумерного изображения трёхмерной сцены?

2. Где физически находится центр проекции при формировании цифрового изображения?

3. Какие из преобразований необратимы и почему?

4. Какие эффекты фотошопа реализованы путём преобразования координат?

5. Какие искажения наблюдаются при выполнении обратного аффинного преобразования в этой работе и почему?

Пример выполнения работы

0	Повернуть изображение относительно центра на заданный угол, уменьшить вдвое и записать в центр результата	$i = x' + 20\ sin\left(\frac{2\pi y'}{128}\right), j = y'$

На форме предусматриваем по кнопке и диалогу для считывания изображения и сохранения каждого из трёх результатов, 4 объекта Tgraphics для хранения и отображения картинок, кнопки для трёх преобразований, поле ввода для угла поворота.

Последовательность действий для преобразования:

1. Загрузка изображения, преобразование в полноцветное.

2. Инициализация нового изображения с теми же параметрами, что и исходное.

3. Ввод угла поворота phi, преобразование в радианы: phi=pi*phi/180

4. Обозначим i, j – координаты пикселей на новом изображении, i1, j1 – на исходном. Так как координаты являются целыми числами, там, где необходимо, будем использовать целочисленное деление div и округление результата round. Для поворота относительно центра надо выполнить перенос в центр: i=i1-width div 2, j=j1-height div 2. Выполнение поворота: i=i1*cos(phi)-j1*sin(phi), j=i1*sin(phi)+j1*cos(phi). Выполнение масштабирования: i=i1 div 2, j=j1 div 2. Выполнение переноса результата в центр: i=i1-width div 2, j=j1-height div 2. Все преобразования можно совместить и записать в обратном виде:

i1=round((((i-width/2)*cos(phi))-((j-height/2)*sin(phi)))*2+width/2),
j1=round((((i-width/2)*sin(phi))+((j-height/2)*cos(phi)))*2+height/2).
Для всех (i,j) результата вычисляем (i1,j1) и, проверяя на попадание координат в пределы изображения, копируем соответствующую точку с исходного изображения на новое.

5. Для восстановления исходного изображения строим обратную функцию и выполняем обратное аффинное преобразование:
i1=round((((i-width/2)*cos(-phi))-((j-height/2)*sin(-phi)))/2+width/2),
j1=round((((i-width/2)*sin(-phi))+((j-height/2)*cos(-phi)))/2+height/2).

6. Строим обратную функцию для нелинейного преобразования и выполняем нелинейные преобразования. Для этого выражаем x' и y' как в системе уравнений: j1=j, i1=i-20*sin(j*pi/64).

7. Предусматриваем возможность записи результатов.

Получаем модуль приблизительно следующего содержания:

```
unit Unit1;

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, FileUtil, Forms, Controls, Graphics, Dialogs, StdCtrls,
  GraphType, IntfGraphics, LCLType, LCLProc, LCLIntf, ExtCtrls, ExtDlgs, Menus,
  fpImage,math,crt;

type

  { TForm1 }
  TForm1 = class(TForm)
    Button1: TButton;
    Button2: TButton;
    Button3: TButton;
    Button4: TButton;
    Button5: TButton;
    Button6: TButton;
    Button7: TButton;
```

```
    Edit3: TEdit;
    Image1: TImage;
    Image2: TImage;
    Image4: TImage;
    Label15: TLabel;
    Label16: TLabel;
    Label17: TLabel;
    Label3: TLabel;
    Label4: TLabel;
    OpenPictureDialog1: TOpenPictureDialog;
    SavePictureDialog1: TSavePictureDialog;
    t: TImage;
    procedure Button1Click(Sender: TObject);
    procedure Button2Click(Sender: TObject);
    procedure Button3Click(Sender: TObject);
    procedure Button4Click(Sender: TObject);
    procedure Button5Click(Sender: TObject);
    procedure Button6Click(Sender: TObject);
    procedure Button7Click(Sender: TObject);
    procedure FormClose(Sender: TObject);
  private
    { private declarations }
  public
    { public declarations }
  end;

var
  Form1: TForm1;
implementation

{$R *.lfm}

{ TForm1 }
//из предыдущих работ
function ConvertTo24Bit(aValue: TBitmap;BackColor: TColor): TBitmap;
begin
  Result := TBitmap.Create;
  with Result do
```

```
    if aValue.PixelFormat <> pf24bit then
      begin
        PixelFormat := pf24bit;
        SetSize(aValue.Width,aValue.Height);
        Canvas.FloodFill(0,0,BackColor,fsBorder);
        Canvas.Draw(0,0,aValue);
      end
    else
      Assign(AValue);
end;

procedure TForm1.Button1Click(Sender: Tobject); //считываем из файла.
begin
  if OpenPictureDialog1.Execute then
    t.Picture.LoadFromFile(OpenPictureDialog1.FileName);
  Label3.Caption:=IntToStr(t.Picture.Bitmap.Width);
  Label4.Caption:=IntToStr(t.Picture.bitmap.Height);
  if t.Picture.bitmap.PixelFormat <> pf24bit then
    begin
      showmessage('invalid pixel color format. trying to convert');
      t.picture.bitmap.assign(convertto24bit(t.Picture.bit-
map,RGB(255,255,255)));
    end;
end;

procedure TForm1.Button2Click(Sender: TObject);
var i,j,i1,j1:integer;phi:real;
begin
phi:=strtoint(edit3.text)/180*pi;//читаем угол, переводим в радианы
with Image4.Picture.Bitmap do//результат — в Image4
  begin
    Clear;
    PixelFormat:=pf24bit;
    Width:=t.Picture.Bitmap.Width;
    Height:=t.Picture.Bitmap.Height;
    Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
    for i:=0 to width-1 do //все пиксели результата
      for j:=0 to height-1 do
```

```
        begin //аффинные преобразования в виде обратных функций.
i1:=round(((i-width div 2)*cos(phi)-(j-height div 2)*sin(phi))*2+width div
2);
j1:=round(((i-width div 2)*sin(phi)+(j-height div 2)*cos(phi))*2+height div
2);
//проверим, не вышли ли за пределы исходного изображения
if i1>=0 then if j1 >=0 then if i1 <Width then if j1 < height then
//переносим пиксель с преобразованными координатами.
 canvas.pixels[i,j]:=t.Picture.Bitmap.Canvas.Pixels[i1,j1];
        end
  end
end;


//процедура записи
procedure TForm1.Button3Click(Sender: TObject);
begin
  if SavePictureDialog1.Execute then
    Image2.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

//нелинейное преобразование координат.
procedure TForm1.Button4Click(Sender: TObject);
var i,j,i1,j1:integer;
begin
with Image1.Picture.Bitmap do //результат — в Image1
  begin
    Clear;
    PixelFormat:=pf24bit;
    Width:=t.Picture.Bitmap.Width;
    Height:=t.Picture.Bitmap.Height;
    Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
    for i:=0 to width-1 do //для всех пикселей результата
      for j:=0 to height-1 do
        begin
         i1:=round(i-20*sin(j*Pi/64));//координаты по обратной функции.
         j1:=round(j);
//Попадают ли вычисленные координаты в исходное изображение.
```

```
        if ((i1>=0) and (j1 >=0) and (i1 <Width) and (j1 < height)) then
canvas.pixels[i,j]:=t.Picture.Bitmap.Canvas.Pixels[i1,j1];
       end
  end
end;

//обратное аффинное преобразование.
procedure TForm1.Button5Click(Sender: TObject);
  var i,j,i1,j1:integer;phi:real;
  begin
  phi:=strtoint(edit3.text)/180*pi;
  with Image2.Picture.Bitmap do //результат — в Image2
    begin
      Clear;
      PixelFormat:=pf24bit;
      Width:=image4.Picture.Bitmap.Width;//источник — в Image4
      Height:=image4.Picture.Bitmap.Height;
      Canvas.FloodFill(0,0,RGB(255,255,255),fsBorder);
      for i:=0 to width-1 do
        for j:=0 to height-1 do
          begin
            i1:=round(((i-width div 2)*cos(-phi)-(j-height div 2)*sin(-
phi))/2+width div 2);
            j1:=round(((i-width div 2)*sin(-phi)+(j-height div 2)*cos(-
phi))/2+height div 2);
            if i1>=0 then if j1 >=0 then if i1 <Width then if j1 < height then
canvas.pixels[i,j]:=image4.Picture.Bitmap.Canvas.Pixels[i1,j1];
          end
    end
  end;

procedure TForm1.Button6Click(Sender: TObject);
begin
if SavePictureDialog1.Execute then
  Image4.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

procedure TForm1.Button7Click(Sender: TObject);
```

```
begin
if SavePictureDialog1.Execute then
  Image1.Picture.SaveToFile(SavePictureDialog1.FileName);
end;

procedure TForm1.FormClose(Sender: TObject);
begin
t.Free;
image1.free;
image2.free;
image4.Free;
end;

end.
```

Результат работы программы (рис. 4):

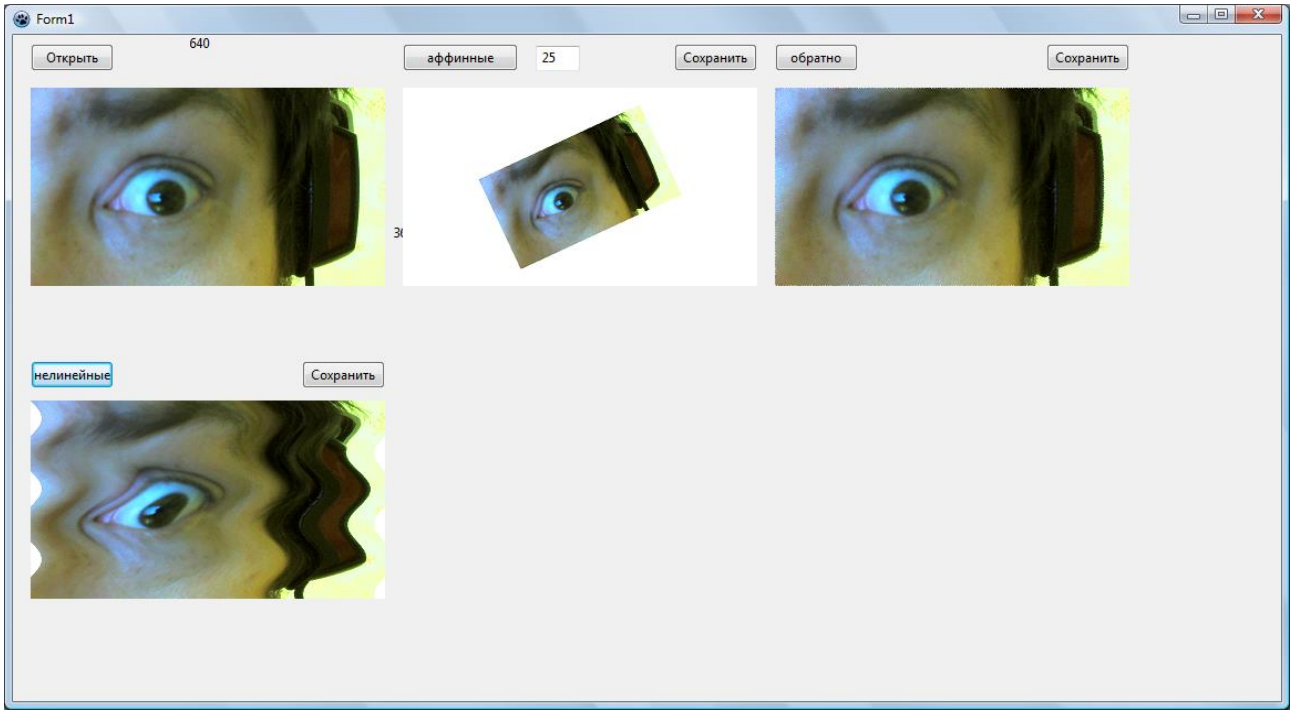


Рис. 4. Результат работы программы геометрических преобразования изображения